

Wipro Capstone Project – Source Code Documentation

Project Name: Guru99Bank Automation Framework

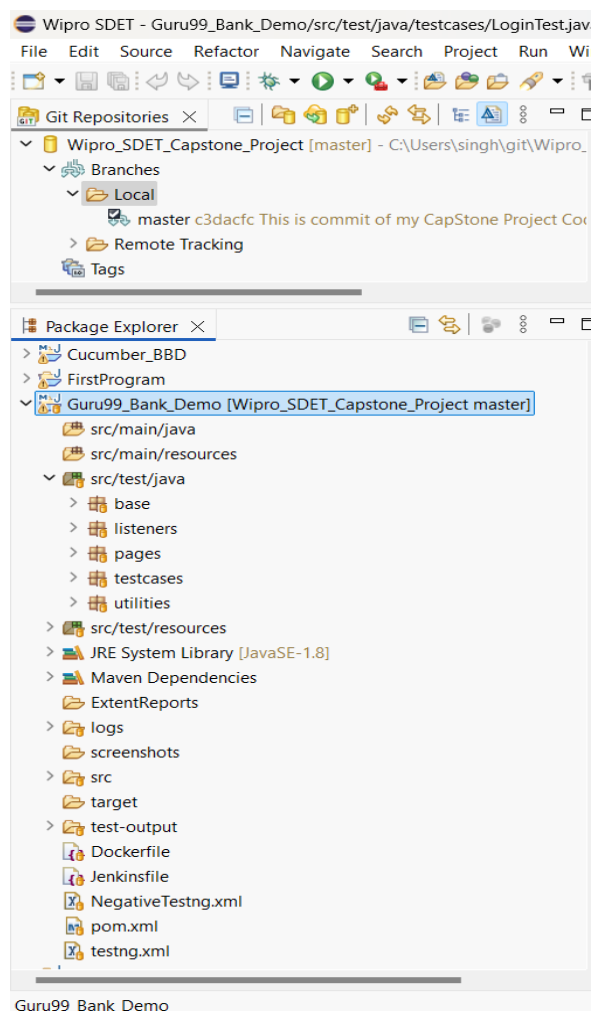
Project Overview

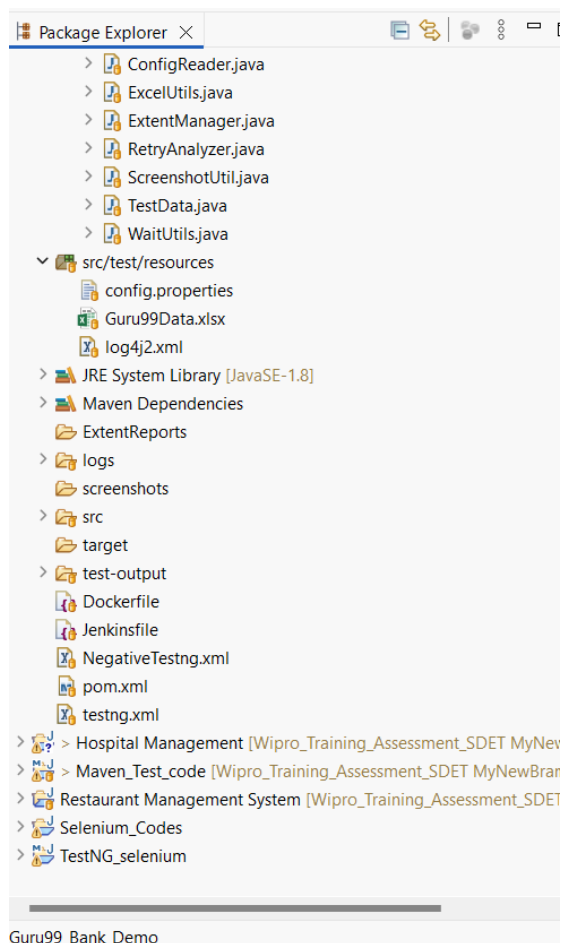
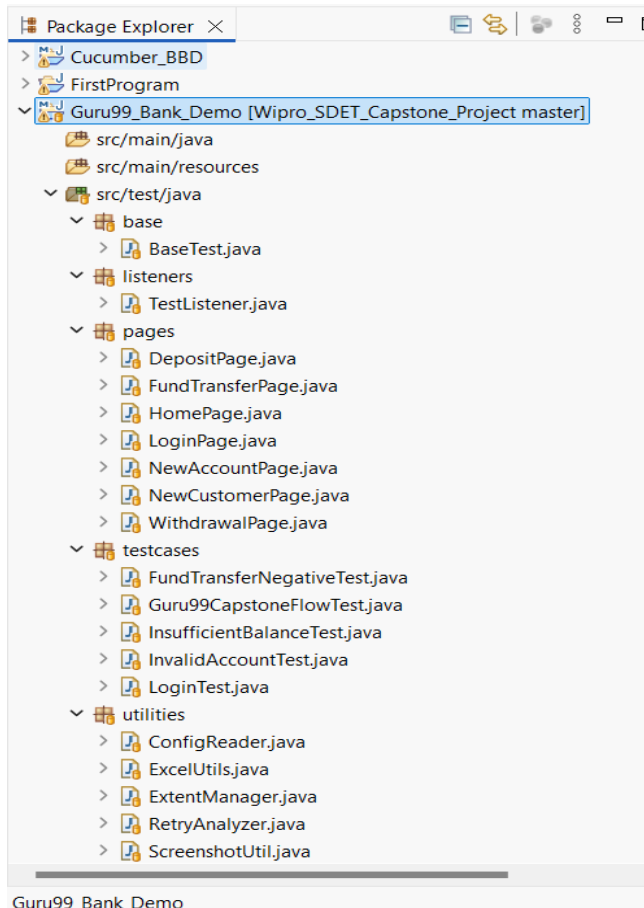
The Guru99 Bank Automation Framework is an end-to-end web automation testing solution built using Java, Selenium WebDriver, TestNG, Maven, Jenkins, Extent Reports, and GitHub, following the Page Object Model (POM) architecture. The framework is designed to automate critical banking operations, including user login, customer registration, account creation, deposits, withdrawals, and fund transfers, along with various validation and negative test scenarios.

To improve maintainability and scalability, the project incorporates reusable page objects, data-driven testing through Apache POI and Excel, automated screenshot capture for failed test cases, retry logic, and TestNG listeners. Comprehensive execution reports are generated using Extent Reports, while Jenkins is utilized for Continuous Integration and automated test execution. The framework is also integrated with Docker and GitHub to support streamlined deployment and version management.

During execution, 5 automated test cases were run successfully, achieving a 100% execution success rate. Additionally, multiple defects were identified, logged, and managed using Jira, showcasing the complete software testing and defect management lifecycle.

Framework Architecture





Project Source Code Details

Package name: base

Class name: BaseTest.java

```
package base;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.chrome.ChromeOptions;
```

```
import org.openqa.selenium.edge.EdgeDriver;
```

```
import org.testng.annotations.AfterMethod;
```

```
import org.testng.annotations.BeforeMethod;
```

```
import io.github.bonigarcia.wdm.WebDriverManager;
```

```
import utilities.ConfigReader;
```

```
public class BaseTest {
```

```
    public static WebDriver driver;
```

```
    ConfigReader config = new ConfigReader();
```

```
    @BeforeMethod
```

```
    public void setup() {
```

```
        String browser = config.getBrowser();
```

```
        if(browser.equalsIgnoreCase("chrome")) {
```

```

WebDriverManager.chromedriver().setup();

ChromeOptions options =
    new ChromeOptions();

// (Headless Run)
options.addArguments("--headless=new");
options.addArguments("--disable-gpu");
options.addArguments("--no-sandbox");
options.addArguments("--disable-dev-shm-usage");
options.addArguments("--remote-allow-origins=*");

// (Normal Browser Run)

//options.addArguments("--remote-allow-origins=*");

driver = new ChromeDriver(options);

}
else if(browser.equalsIgnoreCase("edge")) {

    WebDriverManager.edgedriver().setup();

    driver = new EdgeDriver();
}

driver.manage().window().maximize();

driver.manage().timeouts()
    .implicitlyWait(Duration.ofSeconds(10));

```

```

        driver.get(config.getUrl());
    }

    @AfterMethod
    public void tearDown() {

        try {
            Thread.sleep(2000);
        }
        catch(Exception e) {
            e.printStackTrace();
        }

        if(driver != null) {
            driver.quit();
        }
    }
}

```

Package name: listeners

Class name: TestListener.java

```
package listeners;
```

```
import org.testng.ITestContext;
```

```
import org.testng.ITestListener;
```

```
import org.testng.ITestResult;
```

```
import
```

```
com.aventstack.extentreports.Ext
```

```
entReports;
```

```
import  
com.aventstack.extentreports.Ext  
entTest;
```

```
import base.BaseTest;  
import utilities.ExtentManager;  
import utilities.ScreenshotUtil;
```

```
public class TestListener  
implements ITestListener {
```

```
    ExtentReports extent =
```

```
    ExtentManager.getInstance();
```

```
    ExtentTest test;
```

```
    @Override
```

```
    public void onTestStart(  
        ITestResult result) {
```

```
        test = extent.createTest(  
            result.getName());
```

```
        System.out.println(  
            "STARTED : "  
            + result.getName());
```

```
    }
```

```
    @Override
```

```
public void onTestSuccess(  
    ITestResult result) {  
  
    test.pass("Test Passed");  
  
    System.out.println(  
        "PASSED : "  
        + result.getName());  
}
```

```
@Override
```

```
public void onTestFailure(  
    ITestResult result) {  
  
    test.fail(  
        result.getThrowable());  
  
    try {  
  
        if (BaseTest.driver != null) {
```

```
        ScreenshotUtil.captureScreenshot  
(  
            BaseTest.driver,  
            result.getName());  
        }
```

```
    } catch (Exception e) {
```

```
        System.out.println(
            "Screenshot capture
failed : "
            + e.getMessage());
    }
```

```
        System.out.println(
            "FAILED : "
            + result.getName());

        if (result.getThrowable() !=
null) {
```

```
            System.out.println(
                "Exception Details:");

            result.getThrowable()
                .printStackTrace();
        }
    }
```

```
@Override
public void onTestSkipped(
    ITestResult result) {

    test.skip("Test Skipped");
```

```
        System.out.println(
            "SKIPPED : "
            + result.getName());
```



```
}
```

```
@Override
```

```
public void onStart(
```

```
    ITestContext context) {
```

```
    System.out.println(
```

```
        "Execution Started");
```

```
}
```

```
@Override
```

```
public void onFinish(
```

```
    ITestContext context) {
```

```
    extent.flush();
```

```
    System.out.println(
```

```
        "Execution Completed");
```

```
}
```

```
}
```

Package name: pages

Class name: LoginPage.java

```
package pages;
```

```
import
```

```
org.openqa.selenium.By;
```

```
import
```

```
org.openqa.selenium.WebDriv
```

```
er;
```

```
import utilities.WaitUtils;

public class LoginPage {

    WebDriver driver;

    public LoginPage(WebDriver
driver) {
        this.driver = driver;
    }

    private By userId =
By.name("uid");
    private By password =
By.name("password");
    private By loginBtn =
By.name("btnLogin");

    public void
enterUserId(String uid) {

        WaitUtils.waitForElement(
            driver,
            userId)
            .sendKeys(uid);
    }

    public void
enterPassword(String pass) {

        WaitUtils.waitForElement(
```

```
        driver,  
        password)  
        .sendKeys(pass);  
    }
```

```
public void clickLogin() {
```

```
    WaitUtils.waitForElement(  
        driver,  
        loginBtn)  
        .click();  
}
```

```
public void login(String uid,  
String pass) {
```

```
    enterUserId(uid);  
    enterPassword(pass);  
    clickLogin();  
}  
}
```

Class name: HomePage.java

```
package pages;  
  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;
```

```
public class HomePage {
```

```
    WebDriver driver;
```

```
    public HomePage(WebDriver driver) {
```

```

        this.driver = driver;
    }

    By managerText =

    By.xpath("//*[contains(text(),'Manger
    Id')]");

    public boolean
    verifyManagerHomePage() {

        try {

            return
            driver.findElement(managerText)
                .isDisplayed();

        } catch (Exception e) {

            System.out.println(
                "Manager Id element not
            found");

            return false;
        }
    }
}

```

Class name: NewCustomerPage.java

```
package pages;
```

```
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import utilities.TestData;

import utilities.WaitUtils;


public class NewCustomerPage {

    WebDriver driver;


    public NewCustomerPage(WebDriver
driver) {

        this.driver = driver;

    }


    By newCustomerLink =
By.linkText("New Customer");


    By customerName =
By.name("name");

    By gender =
By.xpath("//input[@value='m']");

    By dob = By.name("dob");

    By address = By.name("addr");

    By city = By.name("city");

    By state = By.name("state");

    By pin = By.name("pinno");

    By mobile = By.name("telephono");

    By email = By.name("emailid");

    By password = By.name("password");
```

```
By submitBtn = By.name("sub");
```

```
public void addCustomer() {
```

```
    WaitUtils.waitForElement(
```

```
        driver,
```

```
        newCustomerLink)
```

```
        .click();
```

```
    driver.findElement(customerName)
```

```
        .sendKeys("Shiv");
```

```
    driver.findElement(gender).click();
```

```
    driver.findElement(dob)
```

```
        .sendKeys("02-10-2001");
```

```
    driver.findElement(address)
```

```
        .sendKeys("Sector 52");
```

```
    driver.findElement(city)
```

```
        .sendKeys("Noida");
```

```
    driver.findElement(state)
```

```
        .sendKeys("Delhi");
```

```
    driver.findElement(pin)
```

```
        .sendKeys("201307");
```

```
    driver.findElement(mobile)
```

```
.sendKeys("9876543210");
```

```
driver.findElement(email)
```

```
.sendKeys("user"
```

```
+
```

```
System.currentTimeMillis()
```

```
+ "@gmail.com");
```

```
driver.findElement(password)
```

```
.sendKeys("Test123");
```

```
WaitUtils.waitForElement(  
    driver,    submitBtn)
```

```
.click();
```

```
String customerId =
```

```
    driver.findElement(  
        By.xpath("//td[text()='Customer  
ID']/following-sibling::td"))
```

```
.getText();
```

```
TestData.customerId =
```

```
customerId;
```

```
System.out.println(  
    "Generated Customer ID =
```

```
"
```

```
+ customerId);
```

```
}  
}
```

Class name: NewAccountPage.java

```
package pages;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.By;
```

```
import
```

```
org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.WebDriver;
```

```
import
```

```
org.openqa.selenium.WebElement;
```

```
import
```

```
org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import
```

```
org.openqa.selenium.support.ui.WebDriverWait;
```

```
import utilities.TestData;
```

```
public class NewAccountPage {
```

```
    WebDriver driver;
```

```
    public NewAccountPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```



```
By newAccountLink = By.linkText("New  
Account");
```

```
By customerId = By.name("cusid");
```

```
By accountType =  
By.name("selaccount");
```

```
By initialDeposit =  
By.name("inideposit");
```

```
By submitBtn = By.name("button2");
```

```
public void createAccount(String  
custId) {
```

```
WebDriverWait wait =  
    new WebDriverWait(driver,  
        Duration.ofSeconds(20));
```

```
WebElement accountMenu =  
    wait.until(  
        ExpectedConditions
```

```
.elementToBeClickable(  
        newAccountLink));
```

```
JavascriptExecutor js =  
    (JavascriptExecutor) driver;
```

```
js.executeScript(  
    "arguments[0].click();",  
    accountMenu);  
  
wait.until(  
    ExpectedConditions  
  
.visibilityOfElementLocated(  
    customerId));  
  
driver.findElement(customerId)  
    .sendKeys(custId);  
  
driver.findElement(accountType)  
    .sendKeys("Savings");  
  
driver.findElement(initialDeposit)  
    .sendKeys("5000");  
  
driver.findElement(submitBtn)  
    .click();  
  
wait.until(  
    ExpectedConditions  
  
.visibilityOfElementLocated(  
    By.xpath(  

```

```
"/td[text()='Account ID']/following-  
sibling::td"))));
```

```
String accountId =  
    driver.findElement(  
        By.xpath(  
            "/td[text()='Account  
ID']/following-sibling::td"))  
        .getText();
```

```
if (TestData.accountId1 == null) {
```

```
    TestData.accountId1 = accountId;
```

```
} else {
```

```
    TestData.accountId2 = accountId;
```

```
}
```

```
System.out.println(  
    "Generated Account ID = "
```

```
    + accountId);  
}
```

```
}
```

Class name: DepositPage.java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;


import utilities.WaitUtils;


public class DepositPage {


    WebDriver driver;


    public DepositPage(WebDriver driver) {
        this.driver = driver;
    }


    By depositLink = By.linkText("Deposit");
    By accountNo = By.name("accountno");
    By amount = By.name("ammount");
    By description = By.name("desc");
    By submit = By.name("AccSubmit");


    public void depositMoney(String accNo, String amt) {


        WebElement depositMenu =
            WaitUtils.waitForClickable(
                driver,
                depositLink);


        JavascriptExecutor js =
            (JavascriptExecutor) driver;


        js.executeScript(
            "arguments[0].click();",
```

```
depositMenu);
```

```
WaitUtils.waitForElement(
```

```
    driver,
```

```
    accountNo)
```

```
    .sendKeys(accNo);
```

```
driver.findElement(amount)
```

```
    .sendKeys(amt);
```

```
driver.findElement(description)
```

```
    .sendKeys("Deposit");
```

```
WaitUtils.waitForClickable(
```

```
    driver,
```

```
    submit)
```

```
    .click();
```

```
}
```

```
}
```

Class name: WithdrawalPage.java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import
```

```
org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.WebDriver;
```

```
import
```

```
org.openqa.selenium.WebElement;
```

```
import utilities.WaitUtils;
```

```
public class WithdrawalPage {

    WebDriver driver;

    public WithdrawalPage(WebDriver
driver) {
        this.driver = driver;
    }

    By withdrawalLink =
By.linkText("Withdrawal");
    By accountNo =
By.name("accountno");
    By amount = By.name("ammount");
    By description = By.name("desc");
    By submit = By.name("AccSubmit");

    public void withdrawMoney(
        String accNo,
        String amt) {

        WebElement withdrawalMenu =
            WaitUtils.waitForClickable(
                driver,
                withdrawalLink);

        JavascriptExecutor js =
            (JavascriptExecutor) driver;

        js.executeScript(
```

```
"arguments[0].click();"
withdrawalMenu);
```

```
WaitUtils.waitForElement(
    driver,
    accountNo)
    .sendKeys(accNo);
```

```
driver.findElement(amount)
    .sendKeys(amt);
```

```
driver.findElement(description)
    .sendKeys("Withdraw");
```

```
WaitUtils.waitForClickable(
    driver,
    submit)
    .click();
```

```
}
```

```
}
```

Class name: FundTransferPage.java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import
org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.WebDriver;
```

```
import
org.openqa.selenium.WebElement;
```

```
import utilities.WaitUtils;
```

```
public class FundTransferPage {

    WebDriver driver;

    public FundTransferPage(WebDriver
driver) {

        this.driver = driver;
    }

    By fundTransferLink =

        By.linkText("Fund Transfer");

    By payerAccount =

        By.name("payersaccount");

    By payeeAccount =

        By.name("payeeaccount");

    By amount =

        By.name("ammount");

    By description =

        By.name("desc");

    By submit =

        By.name("AccSubmit");

    public void transferFunds(

        String fromAcc,

        String toAcc,
```



```

String amt) {

    WebElement fundTransfer =

        WaitUtils.waitForElement(

            driver,

            fundTransferLink);

    JavascriptExecutor js =

        (JavascriptExecutor) driver;

    js.executeScript(

        "arguments[0].click();",

        fundTransfer);

    driver.findElement(payerAccount)

        .sendKeys(fromAcc);

    driver.findElement(payeeAccount)

        .sendKeys(toAcc);

    driver.findElement(amount)

        .sendKeys(amt);

    driver.findElement(description)

        .sendKeys("Fund Transfer");
    WaitUtils.waitForElement(

        driver,

        submit)

        .click();
}
}

```

Package name: testcases

Class name: FundTransferNegativeTest.java

```
package testcases;
```

```
import org.openqa.selenium.By;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseTest;
```

```
import pages.FundTransferPage;
```

```
import pages.LoginPage;
```

```
import pages.NewAccountPage;
```

```
import pages.NewCustomerPage;
```

```
import pages.WithdrawalPage;
```

```
import utilities.ConfigReader;
```

```
import utilities.TestData;
```

```
public class FundTransferNegativeTest
```

```
    extends BaseTest {
```

```
    @Test(priority = 1)
```

```
    public void invalidAccountTransferTest()
```

```
        throws Exception {
```

```
        ConfigReader config =
```

```
            new ConfigReader();
```

```
        LoginPage login =
```

```
            new LoginPage(driver);
```

```
        login.login(
```

```
config.getUsername(),  
config.getPassword());
```

```
NewCustomerPage customer =  
    new NewCustomerPage(driver);
```

```
customer.addCustomer();
```

```
NewAccountPage account =  
    new NewAccountPage(driver);
```

```
account.createAccount(  
    TestData.customerId);
```

```
account.createAccount(  
    TestData.customerId);
```

```
FundTransferPage transfer =  
    new FundTransferPage(driver);
```

```
transfer.transferFunds(  
    "99999999",  
    TestData.accountId2,  
    "100");
```

```
String pageText =  
    driver.findElement(  
        By.tagName("body"))  
        .getText();
```

```
System.out.println(pageText);
```

```
Assert.assertNotNull(pageText);
```

```
System.out.println(  
    "Invalid Account Transfer Executed");  
}
```

```
@Test(priority = 2)  
public void excessiveWithdrawalTest()  
    throws Exception {
```

```
    ConfigReader config =  
        new ConfigReader();
```

```
    LoginPage login =  
        new LoginPage(driver);
```

```
    login.login(  
        config.getUsername(),  
        config.getPassword());
```

```
    NewCustomerPage customer =  
        new NewCustomerPage(driver);
```

```
    customer.addCustomer();
```

```
    NewAccountPage account =  
        new NewAccountPage(driver);
```

```
    account.createAccount(  
        TestData.customerId);
```

```

WithdrawalPage withdrawal =
    new WithdrawalPage(driver);

withdrawal.withdrawMoney(
    TestData.accountId1,
    "100000");

String pageText =
    driver.findElement(
        By.tagName("body"))
        .getText();

System.out.println(pageText);

Assert.assertNotNull(pageText);

System.out.println(
    "Excess Withdrawal Test Executed");
}
}

```

Class name: Guru99CapstoneFlowTest.java

```
package testcases;
```

```
import org.openqa.selenium.By;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseTest;
```

```
import pages.DepositPage;
```

```
import pages.FundTransferPage;
```

```

import pages.HomePage;

import pages.LoginPage;

import pages.NewAccountPage;

import pages.NewCustomerPage;

import pages.WithdrawalPage;

import utilities.ConfigReader;

import utilities.TestData;

import utilities.WaitUtils;


public class Guru99CapstoneFlowTest extends BaseTest {


@Test

public void completeGuru99BankingFlow() throws Exception {


    System.out.println("\n===== GURU99 CAPSTONE PROJECT STARTED
=====\\n");


    ConfigReader config = new ConfigReader();


    LoginPage login = new LoginPage(driver);

    HomePage home = new HomePage(driver);

    NewCustomerPage customer = new NewCustomerPage(driver);

    NewAccountPage account = new NewAccountPage(driver);

    DepositPage deposit = new DepositPage(driver);

    WithdrawalPage withdrawal = new WithdrawalPage(driver);

    FundTransferPage transfer = new FundTransferPage(driver);


    // =====

    // LOGIN

    // =====

```

```
System.out.println("Logging into Guru99 Bank...");
```

```
login.login(  
    config.getUsername(),  
    config.getPassword());
```

```
Thread.sleep(2000);
```

```
Assert.assertTrue(  
    home.verifyManagerHomePage(),  
    "Login Failed");
```

```
System.out.println("Login Successful");
```

```
// =====  
// CREATE CUSTOMER  
// =====
```

```
System.out.println("\nCreating Customer...");
```

```
customer.addCustomer();
```

```
WaitUtils.waitForText(  
    driver,  
    "Customer Registered Successfully");
```

```
Assert.assertTrue(  
    driver.getPageSource()  
        .contains("Customer Registered Successfully"));
```

```
Assert.assertNotNull(
```

```

        TestData.customerId);

System.out.println(
    "Customer ID = "
        + TestData.customerId);

Thread.sleep(2000);

// =====
// CREATE ACCOUNT 1
// =====

System.out.println("\nCreating Account 1...");

account.createAccount(
    TestData.customerId);

Assert.assertNotNull(
    TestData.accountId1);

System.out.println(
    "Account 1 = "
        + TestData.accountId1);

Thread.sleep(2000);

// =====
// CREATE ACCOUNT 2
// =====

System.out.println("\nCreating Account 2...");

```



```
account.createAccount(
    TestData.customerId);

Assert.assertNotNull(
    TestData.accountId2);

System.out.println(
    "Account 2 = "
        + TestData.accountId2);

Thread.sleep(2000);

// =====
// DEPOSIT
// =====

try {

    System.out.println("\nDepositing Money...");

    deposit.depositMoney(
        TestData.accountId1,
        "5000");

    Thread.sleep(2000);

    String pageText =
        driver.findElement(
            By.tagName("body"))
            .getText();
```

```
System.out.println(pageText);

if(pageText.contains("HTTP ERROR 500")) {

    System.out.println(
        "\nAPPLICATION DEFECT FOUND");

    System.out.println(
        "Deposit Functionality Returned HTTP 500");
}
else {

    Assert.assertTrue(
        pageText.contains(
            "Transaction details of Deposit"));

    TestData.account1Balance += 5000;

    System.out.println(
        "Deposit Successful");

    System.out.println(
        "Account 1 Balance = "
            + TestData.account1Balance);
}

} catch(Exception e) {

    System.out.println(
        "Deposit Execution Failed");
```

```
        System.out.println(
            e.getMessage());
    }

    // =====
    // WITHDRAWAL
    // =====

    try {

        System.out.println("\nWithdrawing Money...");

        withdrawal.withdrawMoney(
            TestData.accountId1,
            "1000");

        Thread.sleep(2000);

        String pageText =
            driver.findElement(
                By.tagName("body"))
                .getText();

        System.out.println(pageText);

        if(pageText.contains("HTTP ERROR 500")) {

            System.out.println(
                "Withdrawal Returned HTTP 500");
        }
    }
```

```

else if(pageText.contains(
    "Transaction details of Withdrawal")) {

    TestData.account1Balance -= 1000;

    System.out.println(
        "Withdrawal Successful");

    System.out.println(
        "Balance = "
            + TestData.account1Balance);
    }

} catch(Exception e) {

    System.out.println(
        "Withdrawal Execution Failed");

    System.out.println(
        e.getMessage());
    }

// =====
// FUND TRANSFER
// =====

try {

    System.out.println("\nTransferring Funds...");

    transfer.transferFunds(

```

```
TestData.accountId1,  
TestData.accountId2,  
"2500");
```

```
Thread.sleep(2000);
```

```
String pageText =  
    driver.findElement(  
        By.tagName("body"))  
        .getText();
```

```
System.out.println(pageText);
```

```
if(pageText.contains("HTTP ERROR 500")) {
```

```
    System.out.println(  
        "Fund Transfer Returned HTTP 500");  
}
```

```
else if(pageText.contains(  
    "Fund Transfer Details")) {
```

```
    TestData.account1Balance -= 2500;  
    TestData.account2Balance += 2500;
```

```
    System.out.println(  
        "Fund Transfer Successful");  
}
```

```
} catch(Exception e) {
```

```
    System.out.println(  
        "Fund Transfer Failed");  
}
```

```
"Fund Transfer Execution Failed");
```

```
System.out.println(  
    e.getMessage());  
}
```

```
// =====  
// FINAL SUMMARY  
// =====
```

```
System.out.println(  
    "\n===== FINAL SUMMARY =====");
```

```
System.out.println(  
    "Customer ID : "  
        + TestData.customerId);
```

```
System.out.println(  
    "Account 1 : "  
        + TestData.accountId1);
```

```
System.out.println(  
    "Account 2 : "  
        + TestData.accountId2);
```

```
System.out.println(  
    "Final Account 1 Balance : "  
        + TestData.account1Balance);
```

```
System.out.println(  
    "Final Account 2 Balance : "
```

```
+ TestData.account2Balance);
```

```
System.out.println(
```

```
"\n===== CAPSTONE EXECUTION COMPLETED =====");
```

```
}
```

```
}
```

Class name: LoginTest.java

```
package testcases;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.Alert;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseTest;
```

```
import pages.LoginPage;
```

```
public class LoginTest extends BaseTest {
```

```
    @Test(priority = 1)
```

```
    public void invalidLoginTest() {
```

```
        LoginPage login =  
            new LoginPage(driver);
```

```
        login.login(  
            "wrongUser",  
            "wrongPass");
```

```
        WebDriverWait wait =  
            new WebDriverWait(  
                driver,  
                Duration.ofSeconds(10));
```

```
        Alert alert =  
            wait.until(  
                ExpectedConditions.alertIsPresent());
```

```
        String alertText =  
            alert.getText();
```

```
        System.out.println(
```

```

        "Alert Message : "
        + alertText);

Assert.assertTrue(
    alertText.contains(
        "User or Password is not valid"));

    alert.accept();
}

@Test(priority = 2)

public void emptyLoginTest() {

    LoginPage login =
        new LoginPage(driver);

    login.login("", "");

    try {

        WebDriverWait wait =
            new WebDriverWait(
                driver,
                Duration.ofSeconds(5));

        Alert alert =
            wait.until(
                ExpectedConditions.alertIsPresent());

        String alertText =
            alert.getText();

        System.out.println(
            "Alert Message : "
            + alertText);

        alert.accept();

    } catch (Exception e) {

        System.out.println(
            "No Alert Displayed");

        Assert.assertTrue(
            driver.getCurrentUrl()
                .contains("V4"));

    }
}
}

```


Class name: InvalidAccountTest.java

```
package testcases;

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.DepositPage;
import pages.LoginPage;
import pages.NewAccountPage;
import pages.NewCustomerPage;
import utilities.ConfigReader;
import utilities.TestData;

public class InvalidAccountTest extends BaseTest {

    @Test
    public void invalidAccountDepositTest()
        throws Exception {

        ConfigReader config =
            new ConfigReader();

        LoginPage login =
            new LoginPage(driver);

        login.login(
            config.getUsername(),
            config.getPassword());
    }
}
```

```
NewCustomerPage customer =  
    new NewCustomerPage(driver);  
  
customer.addCustomer();  
  
NewAccountPage account =  
    new NewAccountPage(driver);  
  
account.createAccount(  
    TestData.customerId);  
  
DepositPage deposit =  
    new DepositPage(driver);  
  
deposit.depositMoney(  
    "99999999",  
    "500");  
  
String pageText =  
    driver.findElement(  
        By.tagName("body"))  
        .getText();  
  
System.out.println(pageText);  
  
Assert.assertNotNull(pageText);  
  
System.out.println(  
    "Invalid Account Deposit Test Executed");  
}  
}
```

Class name: InsufficientBalanceTest.java

```
package testcases;
```

```
import org.openqa.selenium.By;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import base.BaseTest;
```

```
import pages.FundTransferPage;
```

```
import pages.LoginPage;
```

```
import pages.NewAccountPage;
```

```
import pages.NewCustomerPage;
```

```
import utilities.ConfigReader;
```

```
import utilities.TestData;
```

```
public class InsufficientBalanceTest
```

```
    extends BaseTest {
```

```
    @Test
```

```
    public void insufficientBalanceTest()
```

```
        throws Exception {
```

```
        ConfigReader config =
```

```
            new ConfigReader();
```

```
        LoginPage login =
```

```
            new LoginPage(driver);
```

```
        login.login(
```

```
            config.getUsername(),
```

```
            config.getPassword());
```

```
NewCustomerPage customer =  
    new NewCustomerPage(driver);
```

```
customer.addCustomer();
```

```
NewAccountPage account =  
    new NewAccountPage(driver);
```

```
account.createAccount(  
    TestData.customerId);
```

```
account.createAccount(  
    TestData.customerId);
```

```
FundTransferPage transfer =  
    new FundTransferPage(driver);
```

```
transfer.transferFunds(  
    TestData.accountId1,  
    TestData.accountId2,  
    "100000");
```

```
String pageText =  
    driver.findElement(  
        By.tagName("body"))  
        .getText();
```

```
System.out.println(pageText);
```

```
Assert.assertNotNull(pageText);
```

```
        System.out.println(
            "Insufficient Balance Test
Executed");
    }
}
```

Package: utilities

Class name: ConfigReader.java

```
package utilities;
```

```
import java.io.FileInputStream;
```

```
import java.util.Properties;
```

```
public class ConfigReader {
```

```
    Properties prop;
```

```
    public ConfigReader() {
```

```
        try {
```

```
            FileInputStream fis =
```

```
                new FileInputStream(
```

```
                    "src/test/resources/config.properties");
```

```
            prop = new Properties();
```

```
            prop.load(fis);
```

```
    } catch (Exception e) {  
  
        e.printStackTrace();  
    }  
}  
  
public String getUrl() {  
  
    return prop.getProperty("url");  
}  
  
public String getBrowser() {  
  
    return prop.getProperty("browser");  
}  
  
public String getUsername() {  
  
    return  
prop.getProperty("username");  
}  
  
public String getPassword() {  
  
    return  
prop.getProperty("password");  
}  
}
```

Class name: ExcelUtils.java

package utilities;

```
import java.io.FileInputStream;
```

```
import
```

```
org.apache.poi.ss.usermodel.DataForm
```

```
atter;
```

```
import
```

```
org.apache.poi.ss.usermodel.Row;
```

```
import
```

```
org.apache.poi.xssf.usermodel.XSSFShe
```

```
et;
```

```
import
```

```
org.apache.poi.xssf.usermodel.XSSFWork
```

```
kbook;
```

```
public class ExcelUtils {
```

```
    public static String getData(
```

```
        String path,
```

```
        int rowNum,
```

```
        int colNum) {
```

```
    String data = "";
```

```
    try {
```

```
        FileInputStream fis =
```

```
            new FileInputStream(path);
```

```
        XSSFWorkbook workbook =
```

```

        new XSSFWorkbook(fis);

XSSFSheet sheet =
    workbook.getSheetAt(0);

Row row =
    sheet.getRow(rowNum);

DataFormatter formatter =
    new DataFormatter();

data =
    formatter.formatCellValue(
        row.getCell(colNum));

workbook.close();

} catch (Exception e) {

    e.printStackTrace();
}

return data;
}

}

Class name: ExtentManager.java

package utilities;

import com.aventstack.extentreports.ExtentReports;

```



```
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
```

```
public class ExtentManager {
```

```
    private static ExtentReports extent;
```

```
    public static ExtentReports getInstance() {
```

```
        if (extent == null) {
```

```
            ExtentSparkReporter spark =
```

```
                new ExtentSparkReporter(
```

```
                    "ExtentReports/ExtentReport.html");
```

```
            spark.config()
```

```
                .setReportName(
```

```
                    "Guru99 Bank Automation Report");
```

```
            spark.config()
```

```
                .setDocumentTitle(
```

```
                    "Execution Report");
```

```
            extent = new ExtentReports();
```

```
            extent.attachReporter(spark);
```

```
            extent.setSystemInfo(
```

```
                "Tester",
```

```
                "Shiv Dayal Singh");
```

```
        }
```

```
        return extent;
    }
}
```

Class name: RetryAnalyzer.java

```
package utilities;
```

```
import org.testng.IRetryAnalyzer;
```

```
import org.testng.ITestResult;
```

```
public class RetryAnalyzer
```

```
    implements IRetryAnalyzer {
```

```
        int count = 0;
```

```
        int maxRetry = 2;
```

```
        @Override
```

```
        public boolean retry(
```

```
            ITestResult result) {
```

```
            if (count < maxRetry) {
```

```
                count++;
```

```
                return true;
```

```
            }
```

```
            return false;
```

```
        }
```

```
    }
```

Class name: ScreenshotUtil.java

```
package utilities;

import java.io.File;
import java.io.IOException;

import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtil {

    public static void captureScreenshot(
        WebDriver driver,
        String fileName) {

        try {

            File src =
                ((TakesScreenshot) driver)
                    .getScreenshotAs(
                        OutputType.FILE);

            String destinationPath =
                System.getProperty("user.dir")
                + "/screenshots/"
                + fileName
                + " _ "
                + System.currentTimeMillis()
                + ".png";
```

```

        File destination =
            new File(destinationPath);

        FileUtils.copyFile(
            src,
            destination);

        System.out.println(
            "Screenshot Saved : "
            + destinationPath);

    } catch(IOException e) {

        e.printStackTrace();

    } catch(Exception e) {

        System.out.println(
            "Screenshot Capture Failed : "
            + e.getMessage());
    }
}
}

```

Class name: TestData.java

```

package utilities;

public class TestData {

    public static String customerId;

    public static String accountId1;

    public static String accountId2;
    public static int account1Balance = 5000;

```

```
    public static int account2Balance = 5000;
}
```

Class name: WaitUtils.java

```
package utilities;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
import
org.openqa.selenium.WebElement;
```

```
import
org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import
org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class WaitUtils {
```

```
    public static WebElement
    waitForElement(
```

```
        WebDriver driver,
```

```
        By locator) {
```

```
        WebDriverWait wait =
```

```
            new WebDriverWait(
```

```
                driver,
```

```
                Duration.ofSeconds(10));
```

```
        return wait.until(
```

```
            ExpectedConditions
```

```
.visibilityOfElementLocated(locator));  
}
```

```
public static WebElement  
waitForClickable(  
    WebDriver driver,  
    By locator) {  
  
    WebDriverWait wait =  
        new WebDriverWait(  
            driver,  
            Duration.ofSeconds(10));  
  
    return wait.until(  
        ExpectedConditions  
        .elementToBeClickable(locator));  
}
```

```
public static void waitForText(  
    WebDriver driver,  
    String text) {  
  
    WebDriverWait wait =  
        new WebDriverWait(  
            driver,  
            Duration.ofSeconds(10));  
  
    wait.until(  
        ExpectedConditions
```

```

.textToBePresentInElementLocated(
    By.tagName("body"),
    text));
}
}

```

Resources

File name: config.properties

url = https://demo.guru99.com/V4/

browser = chrome

username = mngr662638

password = zyjypEb

File name: log4j2.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<Configuration status="WARN">

    <Appenders>

        <Console name="Console"
            target="SYSTEM_OUT">

            <PatternLayout
                pattern="%d [%t] %-5level %logger{36} - %msg%n"/>

        </Console>

        <File name="FileLogger"
            fileName="logs/automation.log">

            <PatternLayout
                pattern="%d %-5p %c - %m%n"/>

        </File>

    </Appenders>

    <Loggers>

        <Root level="info">

            <AppenderRef ref="Console"/>

            <AppenderRef ref="FileLogger"/>

```

```
        </Root>

    </Loggers>

</Configuration>
```

File name: testng.xml

```
<!DOCTYPE suite SYSTEM
"https://testng.org/testng-1.0.dtd">

<suite name="Guru99BankSuite">

    <listeners>

        <listener class-name=
            "Listeners.TestListener"/>

    </listeners>

    <test name="Guru99 Tests">

        <classes>

            <class name="TestCases.LoginTest"/>

            <!-- Customer Module -->
            <class name="TestCases.CustomerTest"/>
            <class name="TestCases.CustomerValidationTest"/>

            <!-- Account Module -->
            <class name="TestCases.AccountTest"/>
            <class name="TestCases.AccountDetailsTest"/>

            <!-- Transaction Module -->
            <class name="TestCases.DepositTest"/>
            <class name="TestCases.WithdrawalTest"/>

            <!-- Fund Transfer Module -->
            <class name="TestCases.FundTransferTest"/>

            <!-- Negative Test Scenarios -->
            <class name="TestCases.InvalidAccountTest"/>
            <class name="TestCases.InsufficientBalanceTest"/>
        </classes>

    </test>

</suite>
```

File name: Jenkinsfile

```
pipeline {
    agent any

    tools {
        maven 'Maven3'
        jdk 'JDK 21'
    }

    stages {
```



```

stage('Checkout Code') {
    steps {
        git branch: 'master',
           url:
'https://github.com/ShivDayalSingh02/Wipro_SDET_Capstone_Project.git'
    }
}

stage('Build Project') {
    steps {
        dir('Guru99_Bank_Demo') {
            bat 'mvn clean compile'
        }
    }
}

stage('Build Docker Image') {
    steps {
        dir('Guru99_Bank_Demo') {
            bat 'docker build -t guru99-bank .'
        }
    }
}

stage('Run Tests in Docker') {
    steps {
        dir('Guru99_Bank_Demo') {
            bat 'docker run --rm guru99-bank'
        }
    }
}

stage('Publish Test Results') {
    steps {
        junit allowEmptyResults: true,
            testResults: 'Guru99_Bank_Demo/target/surefire-reports/*.xml'
    }
}

post {
    always {
        archiveArtifacts(
            artifacts: 'Guru99_Bank_Demo/screenshots/*.png',
            allowEmptyArchive: true
        )

        archiveArtifacts(
            artifacts: 'Guru99_Bank_Demo/logs/*.log',

```

```

        allowEmptyArchive: true
    )

    archiveArtifacts(
        artifacts: 'Guru99_Bank_Demo/ExtentReports/*.html',
        allowEmptyArchive: true
    )

    echo 'Execution Completed'
}

success {

    echo 'Guru99 Bank Automation + Docker Execution Successful'
}

failure {

    echo 'Guru99 Bank Automation + Docker Execution Failed'
}
}
}

```

File name: Dockerfile

```

FROM maven:3.9.9-eclipse-temurin-21

WORKDIR /app

# Install Google Chrome
RUN apt-get update && apt-get install -y \
    wget \
    gnupg \
    unzip \
    && wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | gpg --\
dearmor -o /usr/share/keyrings/google.gpg \
    && echo "deb [arch=amd64 signed-by=/usr/share/keyrings/google.gpg]\
http://dl.google.com/linux/chrome/deb/stable main" >
/etc/apt/sources.list.d/google.list \
    && apt-get update \
    && apt-get install -y google-chrome-stable \
    && rm -rf /var/lib/apt/lists/*

COPY . .

RUN mkdir -p screenshots logs ExtentReports

RUN mvn clean install -DskipTests

CMD ["mvn","clean","test"]

```

File name: NegativeTestng.xml

```

<!DOCTYPE suite SYSTEM
"https://testng.org/testng-1.0.dtd">

<suite name="Guru99NegativeSuite">

    <listeners>

        <listener class-name="Listeners.TestListener"/>
    
```

```

</listeners>

<test name="Negative Tests">

    <classes>

        <class name="TestCases.LoginTest">
            <methods>
                <include name="invalidLoginTest"/>
                <include name="emptyLoginTest"/>
            </methods>
        </class>

        <class name="TestCases.FundTransferNegativeTest"/>

    </classes>

</test>

</suite>

```

File name: pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>Guru99_Bank_Demo</groupId>
    <artifactId>Guru99_Bank_Demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
        <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.12.0</version>
            <scope>test</scope>
        </dependency>
        <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-
java -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.44.0</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.2.5</version>
        </dependency>
        <dependency>
            <groupId>commons-io</groupId>
            <artifactId>commons-io</artifactId>
            <version>2.20.0</version>
        </dependency>
        <dependency>
            <groupId>Cucumber_BBD</groupId>
            <artifactId>Cucumber_BBD</artifactId>
            <version>0.0.1-SNAPSHOT</version>
        </dependency>
        <dependency>
            <groupId>io.github.bonigarcia</groupId>

```

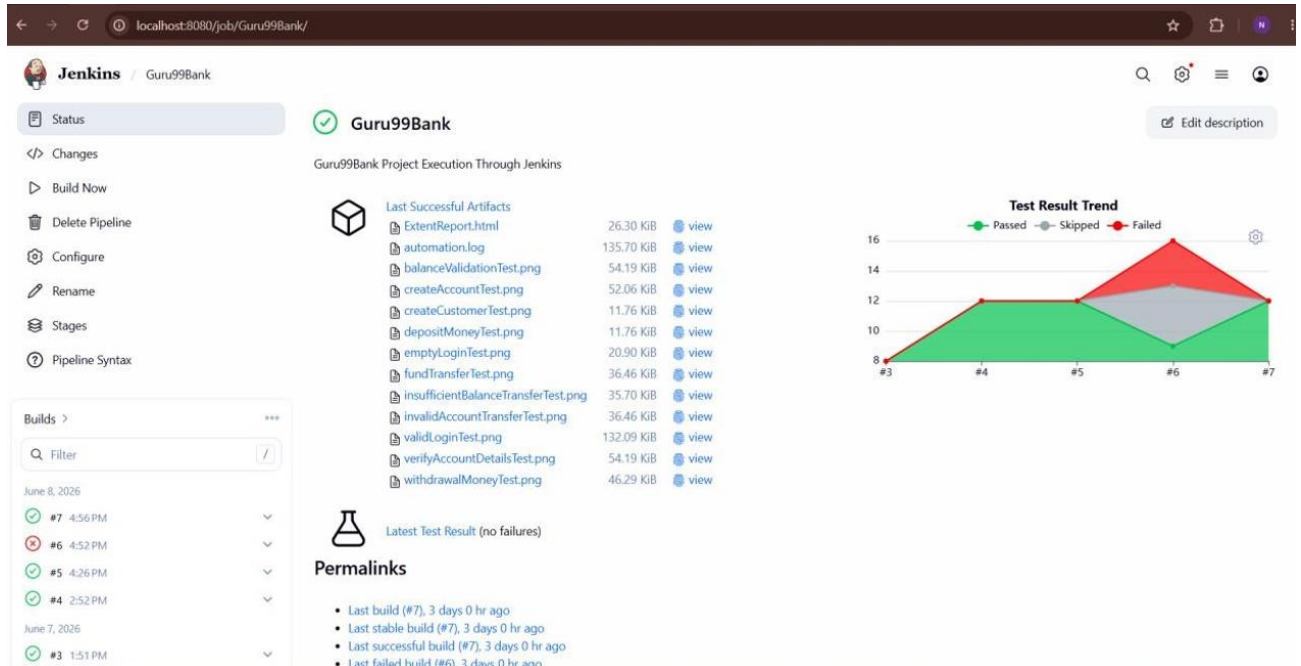
```

        <artifactId>webdrivermanager</artifactId>
        <version>6.1.0</version>
    </dependency>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.23.1</version>
    </dependency>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.23.1</version>
    </dependency>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j2-impl</artifactId>
        <version>2.23.1</version>
    </dependency>
    <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.1</version>
    </dependency>
</dependencies>

    <build>
    <plugins>
    <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.3</version>
    <configuration>
    <suiteXmlFiles>
    <suiteXmlFile>testng.xml</suiteXmlFile>
    </suiteXmlFiles>
    </configuration>
    </plugin>
    </plugins>
    </build>
</project>

```

CI/CD Pipeline Configuration



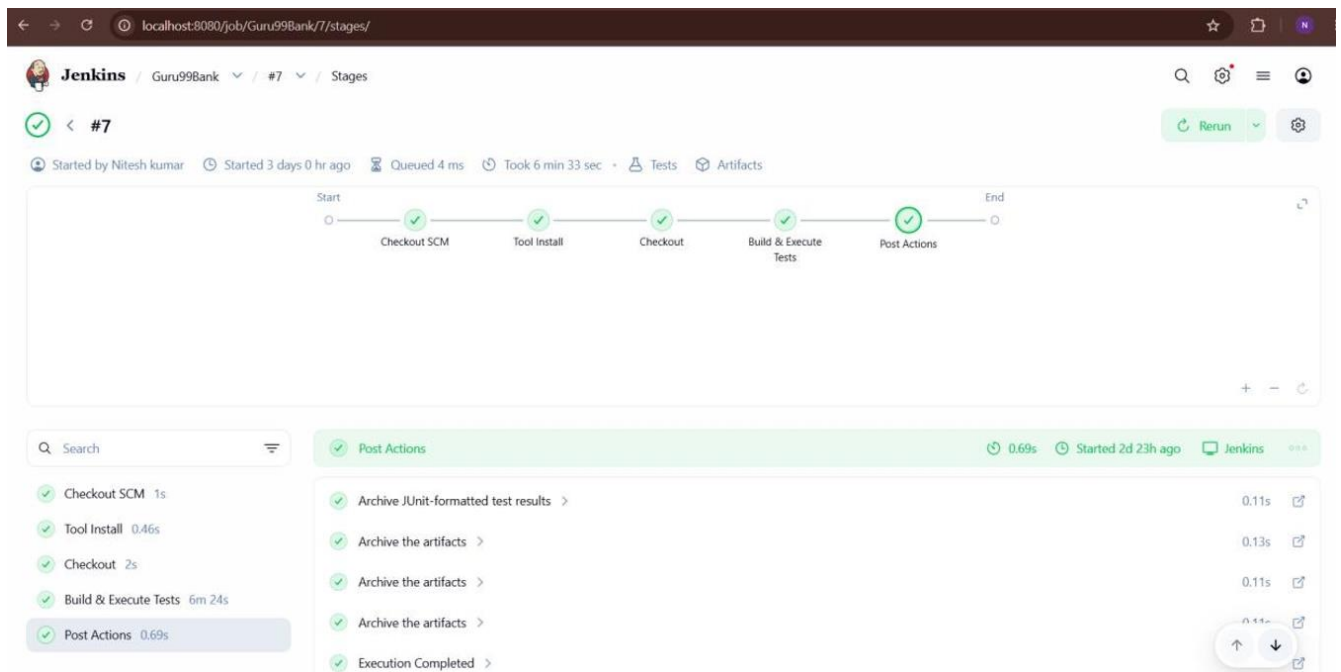
The Jenkins interface for the 'Guru99Bank' pipeline shows a successful build status. The left sidebar contains navigation options: Status, Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main area displays the 'Guru99Bank' pipeline with a description: 'Guru99Bank Project Execution Through Jenkins'. Below this, a list of 'Last Successful Artifacts' is shown, including files like 'ExtentReport.html', 'automation.log', and various test result files (e.g., 'balanceValidationTest.png', 'createAccountTest.png', etc.), each with its size and a 'view' link. A 'Test Result Trend' chart is visible on the right, showing a line graph with 'Passed' (green), 'Skipped' (grey), and 'Failed' (red) data points across builds #3 to #7. The 'Builds' section on the left lists recent builds with their status (green for success, red for failure) and timestamps. The 'Latest Test Result' section indicates 'no failures'. The 'Permalinks' section provides links to the last build, last stable build, last successful build, and last failed build.

Test Result Trend

Build	Passed	Skipped	Failed
#3	8	0	0
#4	12	0	0
#5	12	0	0
#6	10	0	6
#7	12	0	0

Permalinks

- Last build (#7), 3 days 0 hr ago
- Last stable build (#7), 3 days 0 hr ago
- Last successful build (#7), 3 days 0 hr ago
- Last failed build (#6), 3 days 0 hr ago



The Jenkins interface shows the 'Stages' view for the 'Guru99Bank' pipeline, build #7. The top bar indicates the build is successful and provides a 'Rerun' button. The main area displays a horizontal timeline of the build stages: Checkout SCM, Tool Install, Checkout, Build & Execute Tests, and Post Actions. Below the timeline, a search bar and a list of stages are shown. The 'Post Actions' stage is highlighted, showing its duration (0.69s) and start time (2d 23h ago). The 'Post Actions' stage details include 'Archive JUnit-formatted test results' (0.11s), 'Archive the artifacts' (0.13s), 'Archive the artifacts' (0.11s), and 'Execution Completed'.

Stages

- Checkout SCM 1s
- Tool Install 0.46s
- Checkout 2s
- Build & Execute Tests 6m 24s
- Post Actions 0.69s

Post Actions

- Archive JUnit-formatted test results 0.11s
- Archive the artifacts 0.13s
- Archive the artifacts 0.11s
- Execution Completed

Docker Container Configuration

←

→

↺

localhost:8080/job/Guru99Bank_Docker_Execution/

Jenkins

Guru99Bank_Docker_Exce...

🔍

⚙️

☰

👤

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Stages

Pipeline Syntax

Builds >

Filter

Today

🟢 #1 11:30 AM

🟢

Guru99Bank_Docker_Execution

Edit description

Guru99Bank capstone project execution through Docker

Last Successful Artifacts

ExtentReport.html	26.93 KiB	view
automation.log	139.17 KiB	view
balanceValidationTest.png	54.19 KiB	view
createAccountTest.png	52.06 KiB	view
createCustomerTest.png	11.76 KiB	view
depositMoneyTest.png	11.76 KiB	view
emptyLoginTest.png	20.90 KiB	view
fundTransferTest.png	36.46 KiB	view
insufficientBalanceTransferTest.png	35.70 KiB	view
invalidAccountTransferTest.png	36.46 KiB	view
validLoginTest.png	132.09 KiB	view
verifyAccountDetailsTest.png	54.19 KiB	view
withdrawalMoneyTest.png	46.29 KiB	view

Permalinks

- Last build (#1), 5 hr 40 min ago
- Last stable build (#1), 5 hr 40 min ago
- Last successful build (#1), 5 hr 40 min ago
- Last completed build (#1), 5 hr 40 min ago

←

→

↺

localhost:8080/job/Guru99Bank_Docker_Execution/1/stages/

Jenkins

Guru99Bank_Docker_Exce...

🔍

⚙️

☰

👤

#1

Rerun

⚙️

Started by Nitesh kumar

Started 5 hr 41 min ago

Queued 34 ms

Took 2 min 30 sec

Artifacts

Start

Checkout SCM

Tool Install

Checkout Code

Build Project

Build Docker Image

Run Tests in Docker

Publish Test Results

Post Actions

End

🔍 Search

☰

🟢 Post Actions

0.7s

Started 5h 38m ago

Jenkins

...

🟢 Checkout SCM

2s

🟢 Tool Install

0.59s

🟢 Checkout Code

2s

🟢 Build Project

10s

🟢 Build Docker Image

28s

🟢 Run Tests in Docker

1m 31s

🟢 Archive the artifacts

0.19s

🟢 Archive the artifacts

0.11s

🟢 Archive the artifacts

0.12s

🟢 Execution Completed

91ms

🟢 Guru99 Bank Automation + Docker Execution Successful

PERSONAL

Search

Ctrl+K

🔄

🔔 1

📁

⚙️

⋮

N

—

□

➤

🏠 Gordon

🐳 Containers

🐳 Images

📄 Logs

🗄️ Volumes

🌐 Kubernetes

🔧 Builds

🔧 Models

🐳 Docker Hub

🔧 MCP Toolkit BETA

🔧 Extensions

Images [Give feedback](#)

Local My Hub

0 Bytes / 7.7 GB in use 3 Images Last refresh: 0 seconds ago

🔍 Search

☰ ☰

☐		Name	Tag	Image ID	Created	Size	Actions
☐	○	cucumber-framework	latest	d569cb3ecc1e	9 days ago	2.43 GB	🐳 ▶ ⋮ 🗑️
☐	○	nginx	latest	5aca99593157	19 days ago	240.53 MB	🐳 ▶ ⋮ 🗑️
☐	○	guru99-bank	latest	e38e91a9ed19	6 minutes ago	2.4 GB	🐳 ▶ ⋮ 🗑️

Walkthroughs

- FROM node
- RUN mkdir -p
- WORKDIR /app
- COPY package

How do I run a container?

6 mins

Run Docker Hub images

5 mins

View more in the Learning center

Upgrade plan